

Name - Lakshita

Roll no - 2309420008

Course - BTech CSE (DS)

ASSIGNMENT - 4

Short Answer type : Part A

Sol-1) Race Condition (Real - world Example)

A race condition happens when two or more people try to do the same task at the same time, and the final result becomes incorrect because their actions overlap.

Eg : • A piggy bank has ₹ 500.

- Person A and Person B both check it at the same time and see ₹ 500.
- Each decides to take out ₹ 300.
- Both withdraw, and the piggy bank ends up giving ₹ 600, which is incorrect.

This ^{error} happens because they acted simultaneously.

How mutual Exclusion Fixes It
mutual exclusion (mutex) means only one person is allowed to access the shared resource at a time.

Using the same piggy bank example :

- Person A takes a key (lock) to the piggy bank.
- Person B must wait until A finishes.
- A checks ₹ 500 → withdraw ₹ 300 → updates the amount → returns the key.

- Then B checks the new amount (£200) and realizes they cannot withdraw £300.

Thus, the final result is correct.

Sol-2) Comparison : Peterson's Solution vs Semaphores

Aspect	Peterson's Solution	Semaphores
Implementation Complexity	Simple in concept but hard to implement correctly; works only for two processes; requires careful ordering of statements	Easier to use in practice; supports multiple processes; but misuse (like forgetting signal (1)) can cause deadlocks.
Hardware Dependency	Purely software-based and requires no special hardware instructions.	Often relies on atomic hardware instructions (like test-and-set, swap) to ensure atomicity
Scalability	Does not scale well beyond two processes.	Highly scalable for many processes.
Practical Use	Mostly theoretical; rarely used in real systems.	Widely used in operating systems and real applications

Peterson's solution is simple but limited, purely software-based and mainly theoretical. Semaphores are more practical, scalable, and commonly used, but depend on hardware atomic operations.

Sol-3) Advantage of Using Monitors in a Multi-Core System

Monitors provide built-in mutual exclusion, so only one thread can access shared data at a time without manually managing locks. This makes synchronization safer and less error-prone than semaphores. In a multi-core system with many threads running simultaneously, this automatic control greatly simplifies coordination and prevents common mistakes like forgetting to release a semaphore.

Sol-4) Starvation in the Reader-Writer Problem

Starvation occurs when one group (readers or writers) keeps getting delayed forever. For ex; in a readers-priority system, continuous arrival of readers may keep allowing readers to enter, while a waiting writer never gets a chance to write. This leads to writer starvation.

Method to Prevent Starvation

Fair Scheduling (FIFO ordering):

Use a queue to serve readers and writers in the exact order they arrive.

This ensures that once a writer is waiting, no new readers can skip ahead, and every process eventually gets its turn.

Sol-5) Drawback of Eliminating "Hold and Wait"

If a process must request all resources at once before running, it may end up holding resources it won't use until much later. This leads to resource underutilization and longer waiting for other processes.

Example : A process needs a printer only at the end of its execution, but because of this rule, it must request the printer at the beginning. While it runs, the printer stays idle and locked, preventing other processes from printing - even though this process won't use the printer until much later.

Part - B : Application / Numerical Based

Sol-6) Distributed Deadlock Detection Simulation:

Given fragments : $S_1 : P_1 \rightarrow P_2, P_3 \rightarrow P_4$

$S_2 : P_2 \rightarrow P_5, P_5 \rightarrow P_6$

$S_3 : P_6 \rightarrow P_1$

a) Global Wait - For Graph :

$P_1 \rightarrow P_2 \rightarrow P_5 \rightarrow P_6 \rightarrow P_1$

$P_3 \rightarrow P_4$

b) Deadlock Detection :

There is a cycle : $P_1 \rightarrow P_2 \rightarrow P_5 \rightarrow P_6 \rightarrow P_1$

$\therefore$ Deadlock exists, involving processes : P_1, P_2, P_5, P_6

c) Distributed Algorithm to detect it :

Edge - Chasing Algorithm (also called Chandy - Misra - Haas algorithm)
It works by passing probes along wait - for edges to detect cycles across sites.

Sol-7) Distributed File System Performance

a) Expected File Access Time

$$E = (P_{\text{local}} \times T_{\text{local}}) + (P_{\text{remote}} \times T_{\text{remote}})$$

$$E = (0.7 \times 5) + (0.3 \times 25)$$

$$E = 3.5 + 7.5 = 11 \text{ ms}$$

b) Caching strategy : Client - Side Caching (most effective)
Frequently accessed files are stored locally at the client.

- Reduces remote file accesses $\rightarrow$ fewer network delays
- Greatly decreases average access time because local access (5 ms) is much faster than remote (25 ms)

Sol-8) a) optimal mix of checkpoints (10-second period, $RPO=1s$)

- Take incremental checkpoints every 1 second (10 incremental checkpoints).
- If no full checkpoint exists at the start, take one full at $t=0$, then all incrementals.

Total overhead :

- overall incrementals : $10 \times 50 \text{ ms} = 500 \text{ ms}$
- Full + incrementals : $200 \text{ ms} + 500 \text{ ms} = 700 \text{ ms}$

b) Reasoning

- $RPO = 1s \rightarrow$ must checkpoint at least once per second.
- Incremental checkpoints are much cheaper (50 ms vs 200 ms)
- Using only incrementals minimizes total overhead while still meeting RPO.

Sol-9) Case Study - Global E-Commerce Platform :

a) Distributed Scheduling Challenges and Suitable Algorithm

Challenges during flash sales :

- Sudden spike in workload (burst traffic).
- Uneven load distribution across continents.

- Network latency between regions.
- Hotspot servers becoming overloaded.
- Need for quick task migration.

Suitable algorithm :

Dynamic Load Balancing using the least-loaded Algorithm (or Work stealing also acceptable).

B) Fault Tolerance Strategy (RTO + RPO aware)

Strategy :

- Use Geo-replication with active-active data centers + periodic checkpoints.

Justification :

- Active-active ensures near-zero downtime (low RTO) because another region instantly takes over.
- Frequent asynchronous replication + checkpoints provided low RPO, ensuring minimal data loss even if an entire region fails.